

মডিউল ১৭-৩ঃ Fibonacci Series

মডিউল ১৭-৪ঃ Fibonacci Subproblem

মডিউল ১৭-৬ঃ Fibonacci Memoization

মডিউল ১৭-৭ঃ Bottom up approach in Fibonacci series

মডিউল ১৮ঃ Knapsack

মডিউল ১৮-০ঃ ইনট্রডাকশন

মডিউল ১৮-১ঃ Knapsack

মডিউল ১৮-২ঃ Knapsack Recursive Approach

মডিউল ১৮-৩ঃ Knapsack Top Down Approach ইমপ্লিমেন্টেশনসহ

মডিউল ১৮-৫ঃ Knapsack Bottom Up Approach

মডিউল ১৮-৬ঃ Knapsack Bottom Up ইমপ্লিমেন্টেশন

মডিউল ১৯ঃ 0-1 Knapsack Variation

বোনাস মডিউল ২০ঃ Unbounded Knapsack

বোনাস মডিউল ২২ঃ Longest Common Subsequence

বোনাস মডিউল ২৩ঃ Merge Sort

মডিউল ১৮-৩-৪ঃ Knapsack Top Down Approach ইমপ্লিমেন্টেশনসহ

এতক্ষণ আমরা Knapsack problem টিকে recursion দিয়ে সমাধান করেছি, এখন এই প্রব্লেমটির সমাধানকে আরো optimization করতে গেলে আমরা Memoization করে থাকি, এই Memoization পাটটি হচ্ছে DP approach।

```
1 int knapsack(int n, int weight[], int value[], int W)
2
3 {
4
5     if (n == 0 || W == 0)
6
7         return 0;
8
9     if (dp[n][W] != -1)
10
11     {
12
13         return dp[n][W];
14
15     }
16
17     if (weight[n - 1] <= W)
18
19     {
20
21         // duita option
22
23         // niye dekhbo, na niye dekhbo
24
25         int op1 = knapsack(n - 1, weight, value, W - weight[n - 1]) + value[n - 1];
26
27         int op2 = knapsack(n - 1, weight, value, W);
28
29         return dp[n][W] = max(op1, op2);
30
31     }
32
33     else
34
35     {
36
37         // ekta option
38
39         // na niyei dekhite hobe
40
41         int op2 = knapsack(n - 1, weight, value, W);
42
43         return dp[n][W] = op2;
44
45     }
46
47 }
```

আগের কোডটির সাথে যদি compare করা হয়ঃ

```
int knapsack(int n, int weight[], int value[], int W)
{
    if (n == 0 || W == 0)
        return 0;
    if (dp[n][W] != -1)
    {
        return dp[n][W];
    }
    if (weight[n - 1] <= W)
    {
        // duita option
        // niye dekhbo, na niye dekhbo
        int op1 = knapsack(n - 1, weight, value, W - weight[n - 1]) + value[n - 1];
        int op2 = knapsack(n - 1, weight, value, W);
        return dp[n][W] = max(op1, op2);
    }
    else
    {
        // ekta option
        // na niyei dekhite hobe
        int op2 = knapsack(n - 1, weight, value, W);
        return dp[n][W] = op2;
    }
}
```

```
int knapsack(int n, int weight[], int value[], int W)
{
    if (n == 0 || W == 0)
        return 0;
    if (weight[n - 1] <= W)
    {
        // duita option
        // niye dekhbo, na niye dekhbo
        int op1 = knapsack(n - 1, weight, value, W - weight[n - 1]) + value[n - 1];
        int op2 = knapsack(n - 1, weight, value, W);
        return max(op1, op2);
    }
    else
    {
        // ekta option
        // na niyei dekhite hobe
        int op2 = knapsack(n - 1, weight, value, W);
        return op2;
    }
}
```

এখন আমরা recursive call করার আগেই চেক করে নিচ্ছি যে আমাদের 2D array এর মধ্যে আগে থেকে n, W এর অপশনটি store করা আছে কিনা। যদি থাকে তাহলে সেই অপশনটি আবার বের না করে 2D array থাকা অপশনটি return করে দিয়েছি। যদি 2D array এর মধ্যে আগে থেকে n, W এর অপশনটি store না থাকে তাহলে আগের মত recursive call করে op বের করেছি, এখন return করার আগে 2D array এর n, W index এ এখন বের করা অপশনটিকে store করেছি, এরপর সেই অপশনটিকে return করেছি।

এইভাবে Memoization করে আমাদের কোডটির time complexity অনেক কমে যায়।



Adobe Creative Cloud for Teams. Put creativity to work.

Pre-defined templates

Custom

Global variables

value = [4,5,2,5,8,2,1]

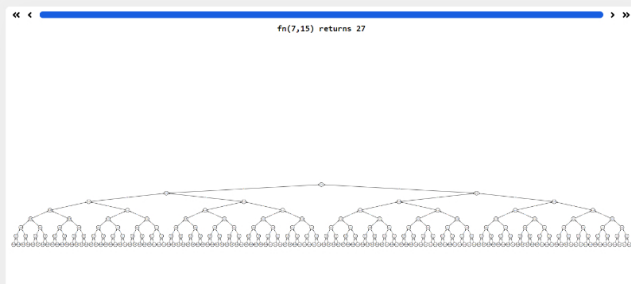
weight = [2,1,3,1,3,1,2]

Recursive function

python

```
def kn(n, W):
    if (n==0 or W==0): return 0

    if (weight[n-1] <= W):
        op1 = kn(n-1, W-weight[n-1]) + value[n-1]
        op2 = kn(n-1, W)
        return max(op1, op2)
    else:
        op2 = kn(n-1, W)
        return op2
```



Options

Step-by-step animation

Memoization

Dark mode

fn(7, 15)

Run

Made with by Bruno Page • Github

উপরে চিত্রে visualize করা input চিতে Memoization ছাড়া করলে এইভাবে এতগুলো কল হয়ে থাকে।

Adobe Creative Cloud for Teams. Put creativity to work.

Pre-defined templates

Custom

Global variables

value = [4,3,2,5,8,2,1]

weight = [2,1,3,1,3,1,2]

Recursive function

python

```
def fn(n, W):
    if n==0 or W==0: return 0
    if (weight[n-1] <= W):
        op1 = fn(n-1, W-weight[n-1])+value[n-1]
        op2 = fn(n-1, W)
        return max(op1, op2)
    else:
        op2 = fn(n-1, W)
        return op2
```

Options

Step-by-step animation

Memoization

Dark mode

fn(7, 15)

Run

Made with by Bruno Page • Github

এরপর Memoization করলে time complexity অনেক কমে যায়। উপরে কালো মার্ক করা n,W কে নতুন করে আবার বের করতে হয় নি, কারন সেইগুলো আগে বের করা হয়েছিল এবং সেইটিকে 2D array তে store করায়, সেইটি আবার নতুন করে বের করতে হয় নি।

সম্পূর্ণ কোডটিঃ

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const int maxN = 1000;
6
7 const int maxW = 1000;
8
9 int dp[maxN][maxW];
10
11 int knapsack(int n, int weight[], int value[], int W)
12 {
13     if (n == 0 || W == 0)
14         return 0;
15     if (dp[n][W] != -1)
16         return dp[n][W];
17     if (weight[n - 1] <= W)
18     {
19         // duita option
20         // niye dekhbo, na niye dekhbo
21         int op1 = knapsack(n - 1, weight, value, W - weight[n - 1]) + value[n - 1];
22         int op2 = knapsack(n - 1, weight, value, W);
23         return dp[n][W] = max(op1, op2);
24     }
25     else
26     {
27         // ekta option
28         // na niyei dekhte hobe
29         int op2 = knapsack(n - 1, weight, value, W);
30         return dp[n][W] = op2;
31     }
32 }
33
34 int main()
35 {
36     int n;
37     cin >> n;
38     int weight[n], value[n];

```

```

68
69     for (int i = 0; i < n; i++)
70     {
71     }
72
73     cin >> weight[i];
74
75     }
76
77     for (int i = 0; i < n; i++)
78     {
79     }
80
81     cin >> value[i];
82
83     }
84
85     int W;
86
87     cin >> W;
88
89     for (int i = 0; i <= n; i++)
90     {
91     {
92
93         for (int j = 0; j <= W; j++)
94         {
95         {
96
97             dp[i][j] = -1;
98
99         }
100     }
101     }
102
103     cout << knapsack(n, weight, value, W) << endl;
104
105     return 0;
106
107 }

```

Time Complexity: $O(n * W)$

<

সিডিউল ১৮-২৪ Knapsack Recursive Approach

Previous

Next

সিডিউল ১৮-৫৪ Knapsack Bottom Up Approach

>

Last updated September 9, 2024

